

Framework for Open-Source Multilingual Large Language Models



Kareem Ibrahim El-desouky

Dr. Ayman Antar Al-Helbawy

Faculty of Computers and Artificial Intelligence

Fayoum University

December 2025

Contents

1. Introduction	5
1.1 Project Scope:	5
1.2. Problem Statement	8
1.3. Research Objectives	9
1.4. Thesis Organization:	11
2. Previous Work	13
2.1 Arabic Language Model Benchmarking	13
2.1.1 LArABench: Comprehensive Arabic AI Benchmarking	13
2.1.2 AlGhafa: Evaluating Arabic Language Models	13
2.2 Arabic Text Understanding and Generation	14
2.3 Sentiment Analysis and Sarcasm Detection in Arabic	14
2.3.1 Evaluating LLMs for Arabic Sentiment Analysis.....	14
2.3.2 GPTaraEval: Evaluating ChatGPT on Arabic NLP	15
2.4 Comprehensive Surveys and Future Directions	15
2.5 Evaluation Metrics and Methodologies	16
2.6 Gaps in Existing Research	17
2.7 Positioning Our Contribution	18
3. Proposed Benchmarking Framework.....	19
3.1. Framework Architecture and Design	19
3.2. Framework Components	22
3.3. Task Specifications.....	28
3.4. Evaluation Methodology	31
4. Implementation.....	36
4.1. Technology Stack and Environment Setup	36
4.2. Data Collection and Preparation.....	40
4.3. Model Selection and Configuration.....	44

4.4. Prompt Engineering Techniques	48
4.5. Benchmark Execution Pipeline	52
5. Case Studies.....	58
5.1. Summarization Task Results	58
5.2. Question Answering Task Results	59
5.3. Sarcasm Detection Task Results	60
5.4. Comparative Analysis.....	61
5.5. Factors Affecting Performance	62
6. Conclusion	66
6.1. Summary of Contributions.....	66
6.2. Key Findings.....	68
6.3. Limitations and Constraints	69
6.4. Impact and Significance	72
6.5. Future Research Trajectory	74
7. References	79

Table of Figures

Figure 1: .env file	39
Figure 2: Poetry install	39
Figure 3: Loading data	41
Figure 4: LLM Client	46
Figure 5: Summarization Prompt	48
Figure 6: QA Prompt	49
Figure 7: Sarcasm Detection Prompt	50
Figure 8: Initialization and Validation	52
Figure 9: Data Preparation	53
Figure 10: Evaluation Loop	53
Figure 11: Results Aggregation	54
Figure 12: Report Generation	54
Figure 13: Summarization Results	58
Figure 14: Question Answering Results	59
Figure 15: Sarcasm Detection Results	60

1. Introduction

If you've used ChatGPT or any AI assistant lately, you've probably been impressed by how they understand and respond to questions. These tools are powered by Large Language Models (LLMs)—essentially very smart computer programs that have learned patterns from massive amounts of text.

But here's the thing: while these models are fantastic at English, they often stumble when it comes to Arabic. And that's a real problem for hundreds of millions of Arabic speakers who deserve the same quality of AI tools that English speakers enjoy.

Think about it—when you're asking an AI to summarize a news article, answer a question about your documents, or understand whether someone's being sarcastic in a social media post, you want it to actually *get* your language and culture, not just translate word-by-word and miss the point entirely.

That's what motivated this project. I wanted to build something practical that could help us understand: **How well do current AI models actually handle Arabic?**

1.1 Project Scope:

This research isn't just about testing models and collecting numbers (though we do plenty of that). It's about creating a tool that anyone can use to evaluate Arabic language AI and then using that tool to figure out where we stand and where we need to go.

The Three Tasks We're Testing

1. Text Summarization

Can AI take a lengthy Arabic article—say, a 500-word news piece—and give you a solid 100-word summary that captures the key points? This isn't just about shortening text; it's about understanding what's actually important.

2. Question Answering

If you give AI a passage of Arabic text and ask a specific question about it, can it find the right answer? This is crucial for everything from student study tools to business document analysis.

3. Sarcasm Detection

Here's where things get culturally interesting. Can AI tell when an Arabic speaker is being sarcastic versus serious? This requires understanding not just words, but context, culture, and tone. We're testing this with both formal Modern Standard Arabic and Egyptian dialect because, let's be honest, that's where a lot of the sarcastic comments happen.

Why This Matters More Than You Might Think

The Language Complexity Challenge

Arabic isn't just another language to process—it's genuinely complex for computers. Words change dramatically based on context. The same root letters can spawn dozens of different words. Plus, Arabic flows right-to-left, has multiple forms for each letter depending on where it appears in a word, and features grammatical intricacies that can trip up even the smartest AI.

The Training Data Problem

Most AI models are trained on whatever text is easiest to find online—and that's predominantly English. Arabic content exists, sure, but it's a much smaller slice of the internet pie. So these models simply haven't seen enough Arabic to truly master it.

The Dialect Dilemma

Unlike English, where regional variations are mostly about accent and a few words, Arabic exists in fundamentally different forms. There's Modern Standard Arabic (MSA) used in formal writing and news broadcasts, but in daily life? People speak Egyptian, Levantine, Gulf, or Maghrebi dialects that can be nearly incomprehensible to each other—and to AI models trained mainly on formal text.

The Cultural Context Gap

When someone posts something sarcastic in Arabic, they're drawing on shared cultural experiences, historical references, and social norms. An AI trained on global data might completely miss the joke, the irony, or the cultural reference that makes the meaning clear to native speakers.

What Makes Our Approach Different

Most studies focus on big commercial models like ChatGPT, which are black boxes you can't modify or fully understand. We're concentrating on open-source models that researchers and developers can actually access, study, and improve.

Plus, we've built everything to be configurable. Don't like our prompts? Write your own. Want to test a different model? Just select it from the dropdown. Do you have your own dataset? Upload it. No coding required.

Comparing Models Side-by-Side

One feature that makes our framework particularly practical is the ability to evaluate multiple models at once. Instead of running separate tests for each model and then trying to compare results manually, you can select up to three models and benchmark them simultaneously on the exact same test data.

Why does this matter? Because when you're trying to decide which AI model to use for your Arabic application, you want to see a clear, apples-to-apples comparison. Our framework ensures that all models see the same questions, same articles, same test cases—so the performance differences you observe are real, not artifacts of different test samples

The results come in two formats: a summary showing key metrics side-by-side (so you can quickly see "Model A got 85% accuracy while Model B got 78%"), and a detailed breakdown showing exactly how each model handled every single test case. This second part is gold for understanding not just which model is better overall, but which model handles certain types of content better—maybe one model is better at formal news articles while another excels at social media text.

1.2. Problem Statement

Despite the impressive capabilities of modern language models, there is a critical gap in how they handle Arabic. The problem manifests in several ways:

The Language Complexity Challenge: Arabic isn't just another language to process—it presents unique computational challenges. Words can have drastically different meanings based on context, diacritical marks, and structure. The language has rich morphology where a single root can generate dozens of related words. Add to this the fact that Arabic is written right-to-left, has different letter forms depending on position, and features complex grammatical rules, and you begin to see why models trained primarily on English struggle.

The Data Scarcity Issue: Most large language models are trained predominantly on English text, simply because that's what's abundantly available online. Arabic content, while substantial, represents a much smaller fraction of training data. This means models inherently have less "experience" with Arabic patterns, idioms, and structures.

The Dialectal Diversity Dimension: Unlike many languages, Arabic exists in multiple forms. Modern Standard Arabic (MSA) is used in formal writing and media across the Arab world. However, in daily conversation, people speak various regional dialects that can differ significantly from MSA and from each other. Egyptian dialect, for instance, has its own vocabulary, expressions, and grammatical patterns that a model trained mainly on MSA might struggle with.

The Cultural Context Gap: Understanding sarcasm, humor, and cultural references requires more than linguistic knowledge—it demands cultural awareness. When an Arabic speaker makes a sarcastic comment, they're drawing on shared cultural experiences, historical references, and social norms. Can AI models trained on global data catch these nuances?

The Evaluation Deficit: Perhaps most critically, there's a lack of standardized, comprehensive evaluation frameworks specifically designed for Arabic NLP tasks. Without proper benchmarking tools, we can't reliably measure progress, compare different approaches, or identify specific areas needing improvement.

The Real-World Impact: This isn't just an academic concern. When Arabic speakers interact with AI assistants, use automated translation tools, or rely on AI-powered search results, they often encounter systems that misunderstand context, miss cultural nuances, or simply provide lower-quality results than their English-speaking counterparts. This represents a form of digital inequality that affects hundreds of millions of people.

Our Research Question: How effectively do open-source multilingual large language models perform on key Arabic natural language processing tasks—specifically summarization, question answering, and sarcasm detection—and how can we develop a comprehensive, configurable benchmarking framework to systematically evaluate and improve their performance?

1.3. Research Objectives

This research has been designed with both immediate practical goals and broader long-term impacts in mind. Here's what we're working to achieve:

1. Build a Robust Benchmarking Framework

- Create a flexible, user-friendly system that anyone can use to test Arabic language models.
- Design it to be modular, so researchers can easily add new models, datasets, prompts or evaluation tasks.
- Ensure reproducibility, anyone should be able to run the same tests and get consistent results.
- Make it accessible through an intuitive web interface that doesn't require deep technical expertise.

2. Conduct Comprehensive Model Evaluation

- Test multiple open-source multilingual models (like GPT-OSS, Qwen, Gemma, and Llama) on Arabic tasks.
- Evaluate performance across different prompt engineering techniques to understand what approaches work best.
- Compare results between Modern Standard Arabic and dialectal Arabic (specifically Egyptian slang).
- Generate both quantitative metrics (hard numbers) and qualitative insights (understanding the "why" behind the results).

3. Identify Strengths and Limitations

- Pinpoint exactly where current models excel in handling Arabic.
- Document specific failure patterns—when and why do models struggle?
- Understand whether failures stem from linguistic issues, cultural gaps, or technical limitations.
- Create a clear picture of the current state of Arabic NLP capabilities.

4. Advance Arabic NLP Research

- Establish standardized benchmarks that the research community can use and build upon.
- Provide empirical evidence about what works and what doesn't in Arabic language processing.
- Contribute datasets, methodologies, and insights to the open-source community.
- Bridge the resource gap between Arabic and more widely studied languages.

5. Enable Practical Improvements

- Offer actionable recommendations for developers working with Arabic language models.
- Provide guidance on prompt engineering strategies that yield better Arabic outputs.
- Identify priority areas where focused research could have the most impact.
- Create tools that make Arabic NLP evaluation accessible to developers and researchers.

Secondary Objectives:

- **Promote Inclusivity in AI:** By highlighting and addressing the disparities in how AI handles different languages, we're advocating for more equitable AI development that serves all language communities.
- **Foster Community Collaboration:** By releasing our framework as open-source software with comprehensive documentation, we're inviting others to contribute, extend, and improve upon this work.

- **Bridge Academia and Practice:** This research is designed to be useful both for academic study and for real-world applications, providing value to researchers and practitioners alike.

Expected Outcomes:

By achieving these objectives, we expect to:

- Provide the first comprehensive open-source benchmarking framework specifically designed for evaluating Arabic capabilities in multilingual LLMs.
- Generate a detailed performance profile of current open-source models on Arabic tasks.
- Create a foundation for future research in Arabic NLP.
- Offer practical tools that developers can immediately use to test and improve Arabic language processing.

1.4. Thesis Organization:

This documentation walks you through the complete journey of this research project, from conceptual design to practical implementation and results. Here's how the content is structured:

Chapter 1: Introduction (Current Chapter)

We've established the foundation by explaining what this project aims to accomplish, why Arabic NLP evaluation matters, the specific problems we're addressing, and the goals we're pursuing. This chapter sets the stage for understanding the motivation and scope of our work.

Chapter 2: Previous Work

This chapter reviews existing research in Arabic NLP, examining key benchmarking efforts like LAraBench and AlGhafa, exploring work on Arabic text understanding and sentiment analysis, and identifying gaps in current research. We position our contribution within this broader context, showing how our framework builds upon and extends previous work.

Chapter 3: Proposed Benchmarking Framework

This chapter describes the theoretical and architectural design of our benchmarking system. We explain how the framework is conceptually organized, what components it comprises, how each

of the three tasks (summarization, question answering, and sarcasm detection) is defined and approached, and what evaluation methodologies we employ to measure model performance accurately and fairly.

Chapter 4: Implementation

Here we transition from design to actual execution. This chapter covers the practical aspects of building the framework: the technologies we used (Python, Streamlit, various NLP libraries), how we collected and prepared our datasets, which models we selected and how we configured them, the prompt engineering techniques we developed and tested, and finally, how the entire benchmark execution pipeline operates from start to finish.

Chapter 5: Case Studies

This is where we present what we discovered through our experiments. We share detailed results for each task—summarization, question answering, and sarcasm detection—showing how different models performed. Beyond individual task results, we provide comparative analysis across models and tasks, examine factors affecting performance, and discuss the implications of our findings.

Chapter 6: Conclusion

We conclude by synthesizing our findings and reflecting on their significance. This chapter summarizes our key contributions to Arabic NLP research, highlights the most important findings from our experiments, honestly discusses the limitations and constraints we encountered, assesses the broader impact of this work, and proposes promising directions for future research that can build upon our foundation.

Chapter 7: References

A comprehensive list of all academic papers, datasets, tools, and resources that informed and supported this research.

2. Previous Work

The field of Natural Language Processing (NLP) has witnessed remarkable progress with the emergence of large language models (LLMs), particularly in their capacity to process multiple languages. However, the development and evaluation of these models for Arabic language tasks have received comparatively less attention despite Arabic being one of the world's most widely spoken languages. This chapter reviews the existing literature on benchmarking and evaluating language models for Arabic NLP tasks, highlighting key contributions that have shaped the current landscape and identifying gaps that our research aims to address.

2.1 Arabic Language Model Benchmarking

2.1.1 LAraBench: Comprehensive Arabic AI Benchmarking

Abdelali et al. (2023) introduced LAraBench[1], a comprehensive benchmark designed to evaluate large language models on a variety of Arabic NLP tasks, including emotion detection, stance detection, and sarcasm detection. The study demonstrated the ability of both proprietary and open-source multilingual models, such as ChatGPT and BLOOMZ, to perform zero- and few-shot learning for Arabic tasks.

LAraBench represented a significant step forward in establishing standardized evaluation protocols for Arabic NLP. The benchmark's strength lies in its comprehensive coverage of diverse tasks and its demonstration that modern LLMs can perform Arabic tasks without extensive fine-tuning. However, LAraBench primarily focused on classification tasks and did not extensively cover generative tasks like summarization or detailed question answering evaluation.

Our proposed framework builds upon LAraBench's foundational work by extending the evaluation scope to include summarization and question answering tasks, providing a more holistic assessment of Arabic NLP capabilities. Additionally, we emphasize configurability and accessibility through an open-source, user-friendly interface that enables researchers without deep technical expertise to conduct rigorous evaluations.

2.1.2 AlGhafa: Evaluating Arabic Language Models

Almazrouei et al. (2023) introduced the AlGhafa[2] Evaluation Benchmark for Arabic Language Models, which focused on evaluating models in tasks such as question answering and text generation. This study highlighted the linguistic complexities inherent in Arabic, especially in pre-trained models.

The ALGhafa benchmark made important contributions by acknowledging and addressing the unique morphological and syntactic challenges that Arabic presents to language models. The research emphasized that Arabic's rich morphology, with its extensive system of roots and patterns, presents distinct challenges compared to morphologically simpler languages like English.

Our work extends ALGhafa's contributions by incorporating sarcasm detection—a task that requires not just linguistic understanding but also cultural and contextual awareness. Furthermore, we prioritize open-source models to ensure reproducibility and accessibility, whereas ALGhafa included both open-source and proprietary models in its evaluation.

2.2 Arabic Text Understanding and Generation

Berrimi[3] (2024) investigated deep learning approaches for understanding and generating Arabic textual data, emphasizing the importance of dataset creation and neural network architectures tailored to the linguistic richness of Arabic.

Berrimi's work provided valuable insights into the architectural considerations necessary for effective Arabic NLP. The research highlighted that simply applying models designed for English to Arabic text often yields suboptimal results due to fundamental linguistic differences. The study advocated for Arabic-specific preprocessing, tokenization strategies, and model architectures that account for the language's unique characteristics.

Our framework leverages these insights by incorporating preprocessing steps specifically designed for Arabic text, including proper handling of right-to-left script, diacritical marks, and morphological variations. Additionally, we ensure our evaluation methodology can handle both formal Modern Standard Arabic (MSA) and informal dialectal variations, such as Egyptian slang, addressing a key concern raised in Berrimi's work about the need for models that perform well across Arabic's diverse linguistic landscape.

2.3 Sentiment Analysis and Sarcasm Detection in Arabic

2.3.1 Evaluating LLMs for Arabic Sentiment Analysis

Khaled[4] et al. (2024) explored the effectiveness of multilingual LLMs for Arabic sentiment and sarcasm detection in their study on evaluating large language models for Arabic sentiment analysis. By employing retrieval-augmented generation techniques, the study showcased the ability of these models to detect nuanced emotions in Arabic text.

This research was particularly significant in demonstrating that modern LLMs can capture subtle emotional and cultural nuances in Arabic text when properly prompted and configured. The study's use of retrieval-augmented generation represented an innovative approach to improving model performance on culturally sensitive tasks.

While Khaled et al.'s work concentrated primarily on sentiment and sarcasm detection, our framework expands the evaluation scope considerably by including summarization and question answering tasks. This broader approach enables us to assess how well models generalize across several types of Arabic NLP challenges, providing a more comprehensive understanding of their capabilities and limitations. Moreover, our systematic evaluation of different prompt engineering techniques allows us to identify which strategies work best for specific task-model combinations.

2.3.2 GPTaraEval: Evaluating ChatGPT on Arabic NLP

Khondaker[5] et al. (2023) analyzed ChatGPT's performance in tasks like irony detection and question answering in GPTaraEval, revealing limitations in the model's ability to handle complex linguistic phenomena in Arabic.

GPTaraEval provided crucial insights into the capabilities and shortcomings of one of the most prominent proprietary language models when applied to Arabic tasks. The study revealed that even highly capable models like ChatGPT exhibit notable weaknesses in handling Arabic's linguistic complexity, particularly in tasks requiring deep cultural understanding like irony detection.

Our research diverges from GPTaraEval's approach by shifting focus exclusively to open-source LLMs. This decision serves multiple purposes: it ensures that our findings are reproducible by other researchers, it promotes transparency in understanding model behavior, and it makes our framework accessible to researchers and developers who may not have access to expensive proprietary APIs. Additionally, by focusing on open-source models, we contribute to the community-driven advancement of Arabic NLP rather than evaluating closed systems that cannot be modified or improved by the broader research community.

2.4 Comprehensive Surveys and Future Directions

Mashaabi[6] et al. (2024) conducted an extensive review in their survey of large language models for Arabic language and its dialects, analyzing the architectures, datasets, and evaluation methodologies employed in Arabic NLP. The survey emphasized challenges in handling dialectal variations and outlined future research directions for addressing these complexities.

This comprehensive survey provided an invaluable overview of the current state of Arabic NLP research, identifying critical gaps and challenges that the field must address. The authors' emphasis on dialectal variations as a major challenge resonated particularly strongly with our research objectives. Arabic exists in numerous dialectal forms—Egyptian, Levantine, Gulf, Maghrebi, and others—each with distinct vocabulary, grammar, and pronunciation patterns. Most language models are trained primarily on Modern Standard Arabic, leaving them ill-equipped to handle dialectal content that dominates everyday communication, social media, and informal contexts.

Our work directly addresses this gap identified by Mashaabi et al. by incorporating Egyptian dialect as a specific case study in our sarcasm detection task. Egyptian Arabic is one of the most widely understood dialects due to Egypt's influential media industry, making it an ideal starting point for dialectal evaluation. By focusing on sarcasm detection in Egyptian slang, we target one of the most culturally nuanced and challenging tasks, requiring models to understand not just linguistic patterns but also cultural references, social contexts, and informal expressions that rarely appear in formal training data.

2.5 Evaluation Metrics and Methodologies

The literature on Arabic NLP evaluation has established several important methodological precedents that inform our work:

Quantitative Metrics: Previous research has established ROUGE for summarization evaluation, BLEU for translation-quality assessment, and METEOR for semantic similarity measurement. These metrics, while originally developed for English, have been successfully adapted for Arabic NLP evaluation, though with recognized limitations in capturing all aspects of Arabic linguistic quality.

Qualitative Analysis: Multiple studies have emphasized that purely quantitative metrics cannot fully capture model performance on Arabic tasks. Cultural appropriateness, dialectal authenticity, and contextual understanding require human evaluation and detailed error analysis—approaches we incorporate into our framework through comprehensive logging and result archiving.

Standardized Datasets: The availability of standardized Arabic datasets for summarization, question answering, and sarcasm detection has been crucial for enabling reproducible research.

Our framework leverages these publicly available resources while also supporting custom dataset uploads, ensuring both standardization and flexibility.

2.6 Gaps in Existing Research

Despite the valuable contributions of previous work, several important gaps remain:

1. **Limited Open-Source Focus:** Much existing research evaluates proprietary models like GPT-3.5 or GPT-4, which are not accessible for detailed study, modification, or local deployment. Our focus on open-source models addresses this gap by providing evaluations that the community can reproduce, verify, and build upon.
2. **Fragmented Evaluation Approaches:** Different studies use different datasets, metrics, and methodologies, making cross-study comparison difficult. We address this by establishing a unified framework with standardized evaluation protocols that can be consistently applied across different models and tasks.
3. **Insufficient Prompt Engineering Analysis:** While prompt engineering has been recognized as crucial for LLM performance, systematic evaluation of different prompting strategies for Arabic tasks remains limited. Our framework explicitly incorporates prompt configurability and comparison as a core feature.
4. **Dialectal Underrepresentation:** Despite acknowledgment that dialectal Arabic presents unique challenges, most benchmarking efforts focus primarily on Modern Standard Arabic. Our inclusion of Egyptian dialect sarcasm detection represents a step toward more comprehensive dialectal evaluation.
5. **Accessibility Barriers:** Many existing benchmarking tools require significant technical expertise to set up and use. Our Streamlit-based web interface dramatically lowers the barrier to entry, enabling researchers, educators, and developers without extensive programming experience to conduct rigorous evaluations.
6. **Limited Task Coverage in Single Frameworks:** Existing frameworks often focus on narrow task sets. By combining summarization, question answering, and sarcasm detection in a single configurable framework, we enable more comprehensive assessment of model capabilities across diverse Arabic NLP challenges.

2.7 Positioning Our Contribution

Our research builds upon and extends this body of previous work in several keyways:

Comprehensive Open-Source Framework: We provide a complete, documented, open-source benchmarking system specifically designed for Arabic NLP evaluation, filling a critical infrastructure gap in the field.

Configurable Architecture: Unlike fixed benchmarks, our modular design allows researchers to easily customize models, datasets, prompts, and evaluation metrics, enabling diverse experimental configurations without code modification.

Balanced Evaluation Scope: By including three distinct tasks—summarization (generative and extractive), question answering (general knowledge), and sarcasm detection (classification and cultural understanding)—we assess different dimensions of language model capability in a single unified framework.

Practical Accessibility: Our user-friendly web interface makes sophisticated NLP evaluation accessible to a broader audience, democratizing access to benchmarking tools and potentially accelerating research progress.

Methodological Rigor with Practical Focus: We maintain scientific rigor through standardized metrics and reproducible methodologies while prioritizing practical utility through features like custom prompt support, flexible dataset handling, and comprehensive result reporting.

Dialectal Awareness: Our explicit inclusion of some Egyptian and Gulf dialect in sarcasm detection acknowledges and begins to address the critical challenge of dialectal variation in Arabic NLP.

The following chapters detail how we translated these objectives into a working system, describe our implementation choices and challenges, present our experimental results, and discuss the implications for Arabic NLP research and development.

3. Proposed Benchmarking Framework

Building an effective benchmarking system requires more than just running tests and collecting scores. It demands thoughtful architectural design, clear component separation, rigorous task definitions, and robust evaluation methodologies. This chapter presents the conceptual foundation of our framework—how we designed it to be flexible, comprehensive, and scientifically sound.

3.1. Framework Architecture and Design

The architecture of our benchmarking framework follows several key design principles that ensure it's both powerful and practical:

Design Philosophy: Keeping Things Modular

Think of our framework like LEGO blocks. Each piece does its own thing, and you can swap pieces in and out without the whole thing falling apart. This wasn't just us being neat freaks—there are real reasons for building it this way:

Separation of Concerns (or: Everyone Stays in Their Lane)

We've organized everything into layers, and each layer has one job:

- 1. Data Layer:** This is your data handler. Whether you're using our Arabic datasets or bringing your own, this layer takes care of the messy work—reading files, checking that everything's formatted correctly, cleaning up the Arabic text, and getting samples ready for testing.
- 2. Model Interface Layer:** Here's where the magic happens. We use OpenRouter/Groq to talk to different models, but here's the cool part—the rest of the framework doesn't need to know or care about the complicated details of each model's API. It just sends a question and gets an answer back. Simple.
- 3. Task Evaluation Layer:** Each of our three tasks (summarization, QA, sarcasm detection) has its own module. These modules know exactly how to format questions for their specific task, understand the answers they get back, and calculate the right metrics. Want to add a new task? Just create a new module—you don't need to touch anything else.

- 4. Reporting Layer:** After all the testing is done, this layer takes the raw numbers and turns them into something useful—JSON files for computer analysis, Excel spreadsheets for human eyes, and nice-looking displays for the web interface.

Configurability at Multiple Levels:

The framework is designed to be configured without code changes. Users can specify:

- Which task to run
- Which model to evaluate
- How many samples to test
- Whether to use custom prompts
- Whether to enable semantic matching for evaluation
- Whether to upload custom datasets

This configuration happens through an intuitive web interface built with Streamlit, making the framework accessible even to users who aren't comfortable with command-line tools or editing code.

The Pipeline Flow:

Understanding how data flows through the framework helps illustrate its design:

1. User Configuration → 2. Data Loading → 3. Prompt Generation →
4. Model Invocation → 5. Response Parsing → 6. Metric Calculation →
7. Result Aggregation → 8. Report Generation

Each step in this pipeline is independent and testable. If something goes wrong, we can pinpoint exactly where the issue occurred. If we want to improve one aspect—say, better prompt templates—we can modify just that step without affecting the others.

Handling Real-World Constraints:

One of our key architectural decisions was acknowledging and working within real-world limitations:

- **API Rate Limiting:** Instead of trying to process everything at once, we built in sequential processing with appropriate delays and error handling.
- **Cost Management:** We provide sample size controls.
- **Failure Recovery:** The framework handles API errors, timeouts, and unexpected responses without crashing and provides API error percentage for failed cases.
- **Progress Tracking:** Real-time progress updates help users understand what's happening during long-running benchmarks.

Extensibility by Design:

While the framework currently supports three tasks and a specific set of models, the architecture makes expansion straightforward:

- Want to add a new task like sentiment analysis? Create a new evaluator module following the established pattern.
- Need to support a different API provider? Implement the model interface for that provider.
- Want to use different evaluation metrics? Add them to the relevant evaluator.
- Have a new dataset? The data loader can accommodate it.
- Also If you want to use a different model, just enter it's name on either Openrouter or Groq to be able to use it.

This extensibility isn't theoretical, it's built into the code structure, making the framework a genuine platform for ongoing research rather than a one-time experiment.

3.2. Framework Components

Now let's explore the specific components that make up the framework and understand what each one does:

1. Configuration Management (config/)

This component stores all the settings and constants that define how the framework operates:

- **benchmark_config.py**: Contains default configurations for each task, including which metrics to compute, timeout values, and API parameters.
- **data_paths.py**: Centralizes all file paths—where to find datasets, where to save logs, where to output reports.

Why separate configuration? It means researchers can modify settings without hunting through code, and it makes it easy to maintain different configurations for different experimental conditions.

2. Data Processing Pipeline (data/)

The data processing component handles two critical functions:

Data Loader (data_loader.py): This module knows how to:

- Read datasets from various formats (CSV, Excel).
- Handle uploaded files from the web interface.
- Validate that datasets contain the required columns for each task.
- Sample specified numbers of examples for testing.
- Manage the default datasets for each task.

The data loader is smart about Arabic text—it understands that the datasets might have different column names, handles right-to-left text properly, and can deal with various text encodings.

Data Preprocessor (data_preprocessor.py): Before text goes to a model, it often needs cleaning and normalization:

- Removing extra whitespace.
- Normalizing Arabic characters (handling different forms of similar letters).

- Ensuring consistent text formatting.
- Preparing text for optimal model processing.

3. Core Services (services/)

These are the "engine room" components that do the heavy lifting:

LLM Client (llm_client.py): This is our gateway to language models. It:

- Manages API authentication and requests.
- Handles rate limiting and retries.
- Formats request according to each model's requirements.
- Parses responses and extracts the relevant text.
- Logs all interactions for debugging and analysis.

The beauty of this component is that it provides a consistent interface regardless of which model you're using. The rest of the framework just asks for completion and receives text back.

Report Service (report_service.py): After benchmarks complete, the report service:

- Aggregates results across all tested examples.
- Calculate summary statistics.
- Generates insights about model performance.
- Prepares data for visualization.

Excel Exporter (excel_exporter.py): This specialized component creates the Excel reports that are perfect for human analysis:

- Each benchmark run gets added as a new row.
- Historical results are preserved, allowing comparison over time.
- Formatting makes the spreadsheets immediately readable.
- Complex nested results are flattened into tabular format.

4. Evaluation Engine (evaluation/)

This is where the actual assessment happens. The evaluation component consists of several modules:

Benchmark Runner (benchmark_runner.py): The orchestrator that:

- Coordinates the entire evaluation process.
- Manages the flow from data loading through result generation.
- Handles progress tracking and status updates.
- Ensures all steps complete successfully.
- Recovers gracefully from errors.

Task-Specific Evaluators: Each task has its own evaluator that understands the unique requirements:

- **Summarization Evaluator (summarization.py):** Computes ROUGE [7] scores (measuring text overlap), BLEU [8] scores (measuring translation-like quality), and METEOR [9] scores (considering synonyms and word order). It knows how to compare generated summaries against reference summaries.
- **QA Evaluator (qa.py):** Calculates accuracy and F1 Score to analyze how well can the model answer the question from the given text passage.
- **Sarcasm Evaluator (sarcasm.py):** Computes classification metrics like accuracy (how often was the model right?), F1 score (balancing precision and recall), and ROC AUC (how well does the model rank examples?). It handles the binary classification task and can work with both hard predictions and confidence scores.

Prompt Builder (prompt_builder.py): This component generates the actual prompts sent to models:

- Loads task-specific prompt templates from markdown files.
- Substitutes variables (like {text}, {question}, {summary}) with actual content.
- Supports custom user-defined prompts.
- Ensures prompts are properly formatted for each model.

Semantic Matcher (`semantic_matcher.py`): An optional but powerful component that uses Google's Gemini [14] to perform semantic evaluation:

- Compares model outputs to reference answers for meaning rather than exact wording.
- Provides confidence scores for each comparison.
- Especially valuable for Arabic where paraphrasing is common.
- Returns match decisions, confidence and detailed reasoning.

Logger (`logger.py`): Maintains detailed logs of everything that happens:

- All model inputs and outputs.
- Timing information.
- Error messages and stack traces.
- Evaluation metrics and scores.

5. Prompt Templates (`prompts/`)

These are the carefully crafted instructions we give to models:

- **`qa.prompt.md`:** Tells the model how to answer questions based on provided context.
- **`summarization.prompt.md`:** Instructs the model on creating concise, accurate summaries.
- **`sarcasm.prompt.md`:** Guides the model in detecting sarcastic language.

These templates use a specific format with placeholders that get filled in with actual content. They're written in natural language that models understand well, and they're easy to modify for experimentation.

6. Application Interface (app/)

The user-facing component that makes everything accessible:

Streamlit [15] Interface (streamlit_benchmark_app.py): Provides a web-based GUI where users can:

- Select tasks and models through dropdown menus.
- Configure benchmark parameters with sliders and inputs.
- Upload custom datasets via drag-and-drop.
- Write custom prompts in text areas.
- View real-time progress as benchmarks run.
- Explore results through interactive tabs and visualizations.

Benchmark Orchestrator (benchmark_orchestrator.py):

The orchestrator has been extended to coordinate evaluation of multiple models:

- **Sequential Execution:** Runs benchmarks for each selected model one after another to respect API rate limits and prevent service throttling.
- **Unified Data Processing:** Ensures all models evaluate the exact same test samples, eliminating sampling variance and enabling fair comparison.
- **Progress Tracking:** Provides real-time updates showing progress across all model evaluations.
- **Result Aggregation:** Combines outputs from multiple models into comparable formats for side-by-side analysis.
- **Model Selection Interface:** Users can select between 1 and 3 models per benchmark run through the Streamlit interface:
 - Support for both preset models (Gemma, GPT-OSS, Qwen, etc.) and custom models.
 - Clear indication of how many models are selected.
 - Validation to ensure at least one model is chosen.

Comparative Reporting Components:

The framework generates two types of unified reports for multi-model benchmarks:

- Comparison Summary Excel: A consolidated spreadsheet showing key metrics (Accuracy, ROUGE-1, ROUGE-L, F1, BLEU, METEOR, etc.) for each model in adjacent columns, enabling immediate visual comparison of overall performance.
- Detailed Samples Excel: A comprehensive file containing all input texts, model outputs, reference answers, and individual scores for every test example. Each row is labeled with the model name, allowing users to filter and analyze how different models handled specific examples.

Visual Comparison Features

The web interface presents multi-model results through:

- Individual Model Tabs: Dedicated tabs for each model showing its complete results.
- Comparison Summary Table: A side-by-side view of all key metrics across models.
- Download Options: Both comparison summary and detailed sample reports are available as Excel files.

Why Multi-Model Comparison Matters: Rather than running separate benchmarks and manually comparing results, researchers can evaluate multiple models simultaneously on identical data. This approach ensures fair comparison, eliminates sampling bias, reduces experimental variance, and dramatically accelerates the model selection process for specific Arabic NLP applications. For practitioners deciding which model to deploy, this feature provides clear, data-driven insights based on head-to-head performance on the same test cases.

3.3. Task Specifications

Each of our three tasks has specific characteristics, requirements, and evaluation approaches. Let's examine them in detail:

Task 1: Arabic Text Summarization

Objective: Can the model read a lengthy Arabic text and produce a shorter version that captures the essential information, main ideas, and key details?

Dataset: We use the Arabic Text Summarization Dataset [10] from Hugging Face (abdalrahmanshahrour/ArabicTextSummarization), containing 8,380 article-summary pairs. These represent real-world Arabic content from various domains.

Input Format: The model receives:

- A full Arabic text (the article or document to be summarized).
- Instructions on creating a good summary.

Expected Output: A coherent, concise Arabic summary that:

- Preserves the main points of the original text.
- Uses natural Arabic language.
- Maintains factual accuracy.
- Achieves appropriate compression (much shorter than the original).

Evaluation Metrics:

1. **ROUGE-1 (Unigram Overlap):** Measures how many individual words appear in both the generated summary and the reference summary. Higher scores indicate better content coverage.
2. **ROUGE-L (Longest Common Subsequence):** Evaluates the longest sequence of words that appears in the same order in both summaries. This captures fluency and structural similarity.
3. **BLEU (Bilingual Evaluation Understudy):** Originally designed for translation, it measures n-gram precision with a brevity penalty. Useful for assessing how closely the generated summary matches reference patterns.

4. **METEOR (Metric for Evaluation of Translation with Explicit Ordering):** A sophisticated metric that considers synonyms, stemming, and word order. Particularly valuable for Arabic with its rich morphology.

Why This Task Matters: Summarization is crucial for information consumption in Arabic-speaking regions. From news aggregation to document processing in business and government, the ability to automatically create accurate summaries in Arabic has significant practical value.

Task 2: Question Answering (QA)

Objective: Given a question about that passage, can the model extract or generate the correct answer?

Dataset: We use ArabicaQA [11] (sadeem-ai/arabic-qna), a machine reading comprehension dataset with 5,000 question-answer pairs based on Arabic passages.

Input Format: The model receives:

- A context passage in Arabic (the source of information).
- A specific question about that passage.
- Instructions to provide an accurate, concise answer.

Expected Output: A precise answer that:

- Directly addresses the question.
- Is grounded in the provided context.
- Uses natural Arabic phrasing.
- Is appropriately concise (neither too brief nor overly elaborate).

Evaluation Metrics:

1. **Exact Match (EM):** The strictest metric—did the model produce the same answer as the reference? This is binary (right or wrong) and leaves no room for near misses.
2. **ROUGE-L:** Measures the similarity between the model's answer and the reference answer, allowing for paraphrasing and minor differences. This provides a softer, more nuanced assessment.

3. **F1 Score:** The harmonic mean of precision.

Why This Task Matters: Question answering is fundamental to many AI applications: virtual assistants answering user queries, educational tools helping students learn, customer service bots providing information, and search engines delivering precise answers rather than just links.

Task 3: Sarcasm Detection

Objective: Can the model recognize when Arabic text is sarcastic rather than literal? This requires understanding cultural context, linguistic cues, and situational irony.

Dataset: We use the Arabic Sarcasm Dataset [12](iabufarha/ar_sarcasm) with 8,440 labeled examples including both Modern Standard Arabic and Egyptian dialectal Arabic.

Input Format: The model receives:

- An Arabic text (typically a social media post or conversational snippet).
- Instructions on identifying sarcastic vs. literal language.

Expected Output: A classification decision:

- "Sarcastic" or "Not Sarcastic".
- Must handle both MSA and Egyptian dialect.

Evaluation Metrics:

1. **Accuracy:** The simplest metric, what percentage of examples did the model classify correctly?
2. **F1 Score:** The harmonic mean of precision (when the model says "sarcastic," how often is it right?) and recall (of all actual sarcastic examples, how many did the model catch?). This is especially important if sarcastic and non-sarcastic examples are imbalanced.
3. **Recall:** Of all truly sarcastic examples, what fraction did the model successfully identify? Important for understanding the model's sensitivity.
4. **ROC AUC (Receiver Operating Characteristic Area Under Curve):** When models provide confidence scores rather than just binary decisions, this measures how well they

rank examples. A perfect score means the model always ranks sarcastic examples higher than non-sarcastic ones.

Why This Task Matters: Sarcasm detection is crucial for:

- Sentiment analysis (misidentifying sarcasm completely reverses sentiment).
- Content moderation (understanding when criticism is joking vs. serious).
- Customer feedback analysis (distinguishing genuine complaints from sarcastic remarks).
- Social media monitoring (interpreting public opinion accurately).

Arabic sarcasm presents unique challenges because it often relies on cultural references, dialectal expressions, and contextual cues that models trained primarily on English might miss.

Cross-Task Considerations:

All three tasks share some common requirements:

- **Arabic Language Proficiency:** Models must handle right-to-left text, complex morphology, and Arabic-specific linguistic features
- **Cultural Awareness:** Especially for sarcasm but relevant to all tasks, understanding cultural context enhances performance
- **Dialectal Variation:** While summarization and QA focus on MSA, sarcasm detection must handle Egyptian dialect
- **Prompt Sensitivity:** How tasks are described in prompts significantly impacts results

3.4. Evaluation Methodology

Our evaluation approach combines rigorous quantitative measurement with qualitative insights, ensuring we capture both the "what" (performance numbers) and the "why" (underlying reasons for success or failure).

Quantitative Evaluation Framework:

Metric Selection Rationale: We didn't choose metrics arbitrarily—each was selected because it captures a specific aspect of performance:

- For summarization, we use multiple overlapping metrics (ROUGE, BLEU, METEOR) because no single metric perfectly captures summary quality. ROUGE [7] emphasizes content coverage, BLEU [8] focuses on precision, and METEOR [9] adds semantic awareness.
- For QA, we combine strict evaluation (Exact Match) with flexible evaluation (ROUGE-L and semantic matching) and F1 because sometimes answers are correct but phrased differently.
- For sarcasm detection, we use multiple classification metrics because accuracy alone can be misleading with imbalanced data, and F1/recall gives us better insight into model behavior.

Evaluation Process:

1. **Sample Selection:** We test on a specified number of examples (configurable, but typically ranging from small samples for quick tests to larger samples for comprehensive evaluation).
2. **Prompt Generation:** For each example, we generate a task-specific prompt using our templates, inserting the relevant text, question, or content to be analyzed.
3. **Model Invocation:** We send the prompt to the selected model via API, handling rate limits, timeouts, and errors appropriately.
4. **Response Parsing:** The model's response is extracted and cleaned. For structured tasks (like sarcasm detection), we parse the response to extract the classification decision.
5. **Metric Calculation:** We compute all relevant metrics by comparing the model's output to the reference answer/summary/label.
6. **Aggregation:** Individual example scores are aggregated to produce overall performance metrics.

Handling Edge Cases:

Real-world evaluation encounters various challenges:

- **API Failures:** When API calls fail, we log the error and continue with remaining examples. Success rate is tracked separately from task performance.
- **Malformed Responses:** If a model returns an answer in an unexpected format, we attempt to parse it gracefully. When parsing fails, we mark it as an error and move on.
- **Timeout Handling:** Long-running requests are capped with timeouts to prevent the benchmark from hanging indefinitely.
- **Rate Limiting:** We respect API rate limits by implementing appropriate delays between requests.

Semantic Evaluation with LLM-as-Judge:

Traditional metrics like Exact Match can be overly strict, especially for Arabic where multiple phrasings can express the same meaning. Our optional semantic matching feature addresses this:

How It Works:

1. We send both the model's answer and the reference answer to Gemini.
2. We ask Gemini [14] to judge whether they're semantically equivalent.
3. Gemini [14] returns a match decision, confidence score, and reasoning.
4. We aggregate these judgments to compute semantic match rate and average confidence.

Benefits:

- Captures meaning rather than just surface form.
- Handles Arabic synonyms and paraphrasing.
- Recognizes dialectal variations.
- Provides interpretable confidence scores.

Limitations:

- Adds API costs and processing time.

- Introduces another model's biases into evaluation.
- Requires Gemini [14] API access.

Qualitative Analysis Approach:

Numbers alone don't tell the complete story. Our framework supports qualitative analysis through:

Detailed Logging: Every model interaction is logged with:

- The exact prompt sent.
- The model's complete response.
- Timing information.
- Computed metrics.
- Any errors or warnings.

This allows researchers to:

- Review specific examples where models succeeded or failed.
- Identify patterns in errors.
- Understand how models interpret prompts.
- Discover edge cases and unusual behaviors.

Result Archiving: Results are saved in multiple formats:

- JSON files contain complete detailed results for programmatic analysis.
- Excel spreadsheets provide historical tracking and easy human review.
- The web interface offers interactive exploration of results.

Comparative Analysis: By running multiple benchmarks with different configurations, researchers can:

- Compare models head-to-head on the same task.
- Test how prompt variations affect performance.

- Evaluate whether certain prompts work better for specific models.
- Understand performance trade-offs.

Statistical Considerations:

Our evaluation methodology acknowledges important statistical principles:

Sample Size: While we support testing small samples for quick experiments, we encourage larger samples for definitive conclusions. Performance can vary across different samples.

Reporting: We report not just average scores but also success rates, error counts, and processing statistics to provide complete transparency.

Continuous Improvement:

The framework's evaluation methodology isn't static. The modular design allows for:

- Adding new metrics as they're developed.
- Incorporating new evaluation techniques.
- Refining existing metrics based on research insights.
- Extending to new tasks with appropriate evaluation approaches.

This flexibility ensures the framework remains relevant as the field of Arabic NLP evolves and as better evaluation methods emerge.

4. Implementation

While the previous chapter described what we designed, this chapter explains how we actually built it. Implementation is where theoretical designs meet practical reality—where we make concrete technology choices, handle real-world constraints, and solve the inevitable challenges that arise when translating ideas into working code.

4.1. Technology Stack and Environment Setup

Building a robust benchmarking framework requires careful selection of technologies that balance power, accessibility, and maintainability. Here's what we chose and why:

Core Programming Language: Python 3.10+

Python was the natural choice for several compelling reasons:

- **Rich NLP Ecosystem:** Python has the most mature libraries for natural language processing, from basic text handling to advanced evaluation metrics.
- **Data Science Integration:** Libraries like pandas make data manipulation straightforward.
- **API Client Libraries:** Excellent support for HTTP requests and API interactions.
- **Community Support:** Vast resources, documentation, and community help for troubleshooting.

Dependency Management: Poetry

Rather than using traditional requirements.txt, we adopted Poetry because it:

- Provides deterministic dependency resolution (everyone gets the same package versions).
- Handles virtual environments automatically.
- Makes the project easy to install with a single command.
- Separates development dependencies from production ones.
- Creates lock files that ensure reproducible builds.

Web Interface: Streamlit

For making the framework accessible, Streamlit [15] was the ideal choice:

- **Rapid Development:** Build interactive web apps with pure Python—no HTML, CSS, or JavaScript required.
- **Natural Python Integration:** Streamlit [15] code reads like regular Python, making it maintainable.
- **Built-in Components:** Dropdowns, sliders, file uploaders, progress bars, and tabs come out of the box.
- **Real-time Updates:** The interface updates immediately as users interact with it..
- **Easy Deployment:** Can be run locally or deployed to cloud services with minimal configuration

Key Libraries and Their Roles:

1. **evaluate (Hugging Face):** Provides standardized implementations of NLP metrics like BLEU [8] and METEOR [9]. We chose this over implementing metrics ourselves because:
 - It's maintained by experts.
 - Results are comparable with other research using the same library.
 - It handles edge cases we might not anticipate.
2. **rouge-score:** The de facto standard for computing ROUGE [7] metrics, essential for summarization and QA evaluation.
3. **scikit-learn:** Supplies classification metrics (accuracy, F1, ROC AUC) for sarcasm detection. It's the gold standard for machine learning metrics in Python.
4. **python-dotenv:** Manages environment variables securely, keeping API keys out of source code.
5. **openpyxl:** Enables Excel file creation and manipulation, crucial for our reporting functionality.
6. **pandas:** The backbone of data manipulation—reading datasets, filtering, sampling, and transforming data.

7. **tqdm**: Provides progress bars for long-running operations, giving users feedback that something is happening.

API Service: OpenRouter

This was a crucial decision driven by practical constraints:

Initial Plan vs. Reality: Originally, we intended to run models locally, which would have provided:

- No API costs.
- Complete control over execution.
- No rate limits.
- Privacy for test data.

Reality Check: Local model execution requires:

- Significant computational resources (high-end GPUs).
- Large storage capacity (models can be tens of gigabytes).
- Complex setup and configuration.
- Technical expertise to troubleshoot issues.

The Solution: OpenRouter[13]/Groq[14] provides a unified API to access multiple language models:

- Single API key works across all models.
- No need for local hardware.
- Simple HTTP requests.
- Consistent interface despite different underlying model APIs.
- Access to models we couldn't run locally (like 70B parameter models).

The Trade-offs: This decision came with constraints:

- Rate limits (requests per minute).
- Token limits (maximum input/output size).

- API costs (though many models have free tiers).
- Dependency on external service availability.

We designed the framework to work within these constraints while still delivering meaningful results.

Environment Configuration:

Setting up the framework requires:

1. **API Keys:** Stored in a .env file:

```
OPENROUTER_API_KEY=your_key_here  
GEMINI_API_KEY=your_key_here # Optional for semantic matching
```

Figure 1: .env file

2. **Python Environment:** Managed by Poetry:

```
poetry install # Installs all dependencies
```

Figure 2: Poetry install

3. **Directory Structure:** Automatically created:

- data/ - Stores datasets
- logs/ - Contains execution logs
- reports/ - Holds generated reports
- reports/excel_results/ - Excel output files
- reports/json_reports/ - JSON detailed results

Development Workflow:

The framework follows modern Python best practices:

- **Virtual Environments:** Poetry creates isolated environments preventing dependency conflicts.
- **Version Control:** Git tracks all changes with meaningful commit messages.

- **Code Organization:** Clear module separation following Python conventions.
- **Documentation:** Inline comments and docstrings explain complex logic.
- **Error Handling:** Try-except blocks catch and handle errors gracefully.

4.2. Data Collection and Preparation

Quality evaluation depends on quality data. Here's how we handle the entire data pipeline from collection through preparation:

Dataset Sources:

We use three publicly available datasets from Hugging Face, chosen for their quality, size, and relevance:

1. Arabic Text Summarization

- **Source:** abdalrahmanshahrour/ArabicTextSummarization on Hugging Face.
- **Size:** 8,380 article-summary pairs.
- **Content:** News articles and other documents with human-written summaries.
- **Language:** Modern Standard Arabic.
- **Quality:** Real-world content with natural summaries.

2. Sadeem Arabic Q/A (Question Answering)

- **Source:** sadeem-ai/arabic-qna on Hugging Face.
- **Size:** Over 5,000 question-answer pairs.
- **Content:** Question answering pairs with their paragraph from Arabic Wikipedia covering diverse topics .
- **Language:** Modern Standard Arabic.
- **Quality:** Curated from Wikipedia with structured QA format

3. Arabic Sarcasm Dataset

- **Source:** iabufarha/ar_sarcasm on Hugging Face.
- **Size:** 8,440 labeled examples.
- **Content:** Social media posts and conversational text.
- **Language:** Mixed MSA and Egyptian dialect.
- **Quality:** Annotated by native Arabic speakers familiar with sarcasm.

Why These Datasets?

Each was selected for specific reasons:

- **Public Availability:** Anyone can access them, ensuring reproducibility.
- **Standard Benchmarks:** Widely used in Arabic NLP research, enabling comparison with other work.
- **Diverse Content:** Cover different genres, styles, and complexity levels.
- **Appropriate Size:** Large enough for meaningful evaluation but manageable given API constraints.
- **Well-Structured:** Clean formatting with consistent column naming.

Data Loading Process:

The DataLoader class handles all data ingestion with flexibility:

```
# Can load from default datasets
loader = DataLoader(task="qa")
data = loader.load_default_dataset()

# Or from custom uploaded files
data = loader.load_from_file(uploaded_file)
```

Figure 3: Loading data

Key Features:

- **Format Detection:** Automatically handles CSV and Excel files.
- **Column Validation:** Ensures required columns exist (e.g., "text" and "summary" for summarization).
- **Error Messages:** Provides clear feedback when data is malformed.
- **Encoding Handling:** Properly reads Arabic text regardless of file encoding.

Preprocessing Pipeline:

Raw data needs preparation before evaluation. The DataPreprocessor handles:

1. Text Normalization:

Arabic text can have inconsistencies that need standardization:

- Different forms of the same letter (ا vs آ vs إ).
- Varied diacritical marks.
- Inconsistent spacing.
- Mixed Arabic and English numerals.

The preprocessor applies normalization rules that:

- Preserve meaning while standardizing form.
- Handle right-to-left text properly.
- Maintain linguistic authenticity.

2. Data Cleaning:

- Remove extraneous whitespace.
- Strip unwanted formatting characters.
- Ensure consistent line endings.
- Handle null or missing values.

Custom Dataset Support:

Understanding that researchers might want to test on their own data, we built in custom dataset support:

Requirements for Custom Datasets:

For **Summarization**:

- Must have columns: text (original document) and summary (reference summary).

For **Question Answering**:

- Must have columns: question, context (passage), and answer.

For **Sarcasm Detection**:

- Must have columns: text (the statement) and sarcasm (0/1 label).

Upload Process:

1. User uploads CSV or Excel file through the web interface.
2. System validates column presence and data types.
3. Data is loaded into the evaluation pipeline.
4. Preprocessing applies automatically.

Data Validation:

Before evaluation begins, we validate:

- **Completeness:** All required fields are present.
- **Format:** Data types are appropriate (text is string, labels are numeric/categorical).
- **Size:** Dataset has sufficient examples for the requested sample size.
- **Encoding:** Text is readable and properly encoded.

If validation fails, users receive specific error messages explaining what needs to be fixed.

Data Privacy Considerations:

When users upload custom datasets:

- Data is processed in memory during the session.
- No permanent storage of uploaded data on servers.
- API calls to model providers follow their privacy policies.

4.3. Model Selection and Configuration

The framework's flexibility in model selection is one of its key strengths. Here's how we approached model configuration:

Supported Models:

We pre-configured several open-source multilingual models known for Arabic support:

1. Google Gemma-3-27B

- **Parameters:** 27 billion.
- **Strengths:** Strong instruction following, good multilingual capabilities.
- **Arabic Performance:** Competitive on formal MSA tasks.
- **Availability:** Free tier through OpenRouter.

2. GPT-OSS-20B

- **Parameters:** 20 billion.
- **Strengths:** High-capacity open-source model with strong reasoning and multilingual comprehension abilities.
- **Arabic Performance:** Demonstrates notable fluency and understanding across complex Arabic NLP tasks, including summarization and question answering.
- **Availability:** Accessible via OpenRouter/Groq, providing convenient API-based integration for research and benchmarking purposes.

3. Qwen3-4B

- **Parameters:** 4 billion.
- **Strengths:** Efficient, fast inference despite smaller size.
- **Arabic Performance:** Surprisingly capable given the size.
- **Availability:** Free tier through OpenRouter.

4. Meta Llama-3.3-70B-Instruct

- **Parameters:** 70 billion.
- **Strengths:** State-of-the-art performance, excellent instruction following.
- **Arabic Performance:** Among the best for Arabic NLP.
- **Availability:** Free tier through OpenRouter.

Why These Models?

Selection criteria included:

- **Multilingual Training:** All models were trained on diverse language data including Arabic.
- **Open Source:** Code and model weights are openly available.
- **API Access:** Available through OpenRouter[13] / Groq[16] for practical testing.
- **Size Variety:** Range from 4B to 70B parameters, showing how scale affects performance.
- **Recent Releases:** All are relatively new models incorporating latest advances.

Custom Model Support:

Users aren't limited to our predefined list. The framework supports any model available through OpenRouter:

- Select "Other (custom)" in the model dropdown.
- Enter the model identifier (e.g., "anthropic/claude-3.5-sonnet").
- The framework automatically uses that model.

Model Configuration via LLMClient:

The LLMClient class abstracts model interaction:

```
client = LLMClient(  
    api_key=api_key,  
    model_name=model_name,  
    max_tokens=2048,  
    temperature=0.3  
)  
response = client.get_completion(prompt)
```

Figure 4: LLM Client

Error Handling:

Model interactions can fail for several reasons. We handle:

- **API Errors:** Network issues, server problems, authentication failures.
- **Rate Limiting:** Too many requests too quickly.
- **Timeout:** Model takes too long to respond.
- **Malformed Responses:** Model returns unexpected format.

For each error type, we:

- Log detailed error information.
- Increment error counters.
- Continue with remaining examples.
- Report success rate in final results.

Response Parsing:

Models can return responses in various formats. Our parsing logic:

- Extracts the actual answer/summary/classification from the response.
- Handles extra text (like "The answer is: ...").

- Strips formatting characters.
- Validates output format.

For sarcasm detection specifically:

- Looks for clear indicators like "Sarcastic", "Not Sarcastic", "Yes", "No".
- Handles various phrasings the model might use.
- Falls back to binary detection if format is ambiguous.

4.4. Prompt Engineering Techniques

How we ask questions dramatically affects what answers we get. Prompt engineering—crafting effective instructions for language models—is both art and science. Here's our approach:

Prompt Template Design:

Each task has a carefully designed template stored in markdown files:

Summarization Prompt Structure:

You are an expert Arabic text summarization assistant. Your task is to generate clear, concise summaries that capture the essential information while discarding unnecessary details.

Text to summarize:

""{text}""

Instructions:

- Write the summary in Modern Standard Arabic (MSA).
- Keep it concise: maximum 3 sentences.
- Extract and preserve only the main ideas and key facts.
- Do not add information not present in the original text.
- Ensure the summary is coherent, natural, and grammatically correct.
- Remove redundant details, repetitions, and filler content.
- Maintain objectivity - do not insert opinions or interpretations.

Summary:

Figure 5: Summarization Prompt

Design Principles:

- **Clear Role:** Tell the model what expertise it should assume.

- **Explicit Task:** State exactly what we want.
- **Format Guidance:** Specify output format (summary only, no extras).
- **Language Consistency:** Request Arabic output for Arabic input.

Question Answering Prompt Structure:

You are a specialized question-answering assistant that extracts answers from Arabic passages.

Passage:

""{text}""

Question:

"{question}"

Instructions:

- Answer in Modern Standard Arabic (MSA) only.
- Extract the answer directly from the passage provided - do not use external knowledge.
- If the answer exists or can be reliably inferred from the passage, provide a concise, clear response.
- If the passage lacks sufficient information to answer accurately, respond with exactly: "لا توجد معلومات كافية للإجابة".
- Do not fabricate details or add information not present in the passage.
- Keep your answer as brief as possible - use the minimum words necessary for correctness.
- Match the linguistic style and terminology used in the passage.

Answer:

Figure 6: QA Prompt

Design Principles:

- **Sequential Instructions:** Read first, then answer.
- **Constraint:** Answer must come from context (avoid hallucination).
- **Brevity:** Request concise answers to avoid rambling.

Sarcasm Detection Prompt Structure:

You are a specialized classifier that detects sarcasm in Arabic text.

Text:

"{text}"

Instructions:

- Analyze the text carefully for sarcasm indicators such as:
 - Exaggerated praise that implies criticism
 - Statements that contradict obvious reality
 - Ironic tone or unexpected word choices
 - Context that suggests the opposite of literal meaning
- Respond with ONLY one of these labels:
 - "ساخر" (sarcastic)
 - "غير ساخر" (not sarcastic)
- Output only the label with no explanation or additional text.

Label:

Figure 7: Sarcasm Detection Prompt

Design Principles:

- **Binary Output:** Force clear classification.
- **Cultural Awareness:** Acknowledge this requires language expertise.
- **Simplicity:** No complex reasoning required in output.

Custom Prompt Support:

Recognizing that optimal prompts vary by model and task, we allow customization:

How It Works:

1. User toggles "Use Custom Prompt Template" in the interface.
2. Text area appears for entering custom prompt.
3. Users write their own instructions, using the same placeholder variables.
4. Framework uses custom prompt instead of default.

Use Cases:

- **Research:** Testing how prompt variations affect performance.
- **Optimization:** Finding the best prompt for a specific model.
- **Specialization:** Adding domain-specific instructions.
- **Language Variants:** Adapting prompts for dialectal differences.

Prompt Engineering Experiments:

Our framework enables systematic prompt testing:

Paraphrasing:

- Write multiple versions of the same instruction.
- Test each version separately.
- Compare results to identify optimal phrasing.

Instruction Detail:

- Test minimal prompts vs. detailed instructions.
- Measure whether more guidance helps or hurts.
- Find the sweet spot between brevity and clarity.

Few-Shot Examples:

- Include example input-output pairs in prompts.
- Test zero-shot (no examples) vs. few-shot (with examples).
- Measure impact on performance.

4.5. Benchmark Execution Pipeline

This is where everything comes together. The execution pipeline coordinates all components to run complete benchmarks from start to finish.

Pipeline Orchestration:

The BenchmarkOrchestrator class manages the entire process:

Phase 1: Initialization and Validation

1. Load configuration
2. Validate API keys
3. Check task selection
4. Verify model availability
5. Prepare logging

Figure 8: Initialization and Validation

Phase 2: Data Preparation

6. Load dataset (default or custom)
7. Validate data format
8. Sample requested number of examples
9. Preprocess text
10. Log dataset statistics

Figure 9: Data Preparation

Phase 3: Evaluation Loop

- For each example in sample:
11. Generate prompt from template
 12. Send request to model via API
 13. Parse model response
 14. Compute metrics
 15. Update progress
 16. Handle any errors
 17. Log interaction

Figure 10: Evaluation Loop

Phase 4: Results Aggregation

18. Calculate aggregate metrics
19. Compute success rates
20. Generate statistics
21. Perform semantic matching (if enabled)
22. Compile detailed results

Figure 11: Results Aggregation

Phase 5: Report Generation

23. Create JSON report
24. Generate Excel spreadsheet
25. Format for UI display
26. Archive results
27. Clean up temporary data

Figure 12: Report Generation

Real-Time Progress Tracking:

Users see live updates through the Streamlit [15] interface:

Progress Bar:

- Shows percentage completion (0-100%).
- Updates after each processed example.
- Visual indicator of benchmark progress.

Stage Indicators:

- Initialization → Data Loading → Processing → Evaluation → Report Generation.
- Each stage has distinct visual representation.

- Current stage is prominently displayed.

Error Recovery Strategies:

When things go wrong, the framework handles it gracefully:

API Request Failures:

- Catch network errors and API exceptions.
- Log the specific error.
- Mark example as failed.
- Continue with next example.
- Include failure count in final statistics.

Response Parsing Errors:

- Attempt multiple parsing strategies.
- Log unparsable response for review.
- Mark as error but continue.
- Provide examples of failures in report.

Rate Limit Handling:

- Detect rate limit responses.
- Pause execution briefly.
- Retry the request.
- Adjust timing to avoid future limits.

Timeout Management:

- Set reasonable timeouts per request.
- Cancel hanging requests.
- Log timeout events.

- Move to next example.

Semantic Matching Integration:

When enabled, semantic matching adds an additional evaluation layer:

Process:

1. After getting model response and computing standard metrics.
2. Send both model answer and reference answer to Gemini.
3. Request semantic equivalence judgment.
4. Receive match decision, confidence, and reasoning.
5. Log semantic evaluation results.
6. Include in aggregate statistics.

Result Persistence:

Results are saved immediately and incrementally:

JSON Reports:

- Complete detailed results in structured format.
- Saved as {task}_last_run_archive.json.
- Contains every example, metric, and metadata.
- Perfect for programmatic analysis.

Excel Spreadsheets:

- Summary view optimized for human review.
- New runs append as new rows.
- Historical tracking enabled.
- Saved as {task}_results.xlsx.

Log Files:

- Timestamped execution logs.
- All model interactions recorded.
- Error traces for debugging.
- Saved in logs/ directory.

Post-Execution Analysis:

After completion, the framework provides:

- **Summary Statistics:** Key metrics briefly.
- **Detailed Breakdowns:** Per-example results available.
- **Semantic Analysis:** If enabled, detailed matching insights.
- **Performance Metrics:** Processing time, success rates, error analysis.

Web Interface Integration:

The Streamlit [15] UI provides:

- **Tabbed Results View:** Summary, Detailed Scores, Statistics, Semantic Analysis.
- **Interactive Exploration:** Click through different result aspects.
- **Download Options:** Export results for external analysis.
- **Visual Feedback:** Color-coded metrics, success indicators, warning messages.

This complete pipeline ensures that running a benchmark is straightforward while maintaining scientific rigor and providing comprehensive insights into model performance.

5. Case Studies

This chapter presents the empirical findings from our evaluation of open-source multilingual language models on Arabic NLP tasks, building upon previous benchmarking efforts in Arabic NLP [1, 2, 5]. We conducted experiments on a sample of 1000 examples for each task, testing how different models perform on summarization, question answering, and sarcasm detection. While this represents an initial exploration rather than an exhaustive evaluation, the results provide valuable insights into model capabilities and areas for improvement. Our evaluation builds upon previous benchmarking efforts in Arabic NLP [1, 2, 5, 6], while focusing specifically on open-source multilingual models using standardized datasets [10, 11, 12] and established evaluation metrics [7, 8, 9].

5.1. Summarization Task Results

We evaluated three models on their ability to generate concise, accurate Arabic summaries from longer texts.

Evaluation Setup:

- **Dataset:** Arabic Text Summarization (abdalrahmanshahrour/ArabicTextSummarization).
- **Sample Size:** 1000 examples.
- **Models Tested:** GPT-OSS-20B, Gemma-3-27B, Qwen-3-4B.
- **Evaluation Metrics:** ROUGE-1, ROUGE-L, BLEU, METEOR.

Performance Results:

Model	ROUGE-1	ROUGE-L	BLEU	METEOR
GPT-OSS-20B	0.412	0.378	0.148	0.329
Gemma-3-27B	0.371	0.341	0.136	0.306
Qwen-3-4B	0.349	0.318	0.124	0.285

Figure 13: Summarization Results

Key Findings:

GPT-OSS-20B demonstrated the strongest summarization capabilities across all metrics:

- Achieved higher ROUGE-1 score than Gemma-3-27B, indicating better content coverage.
- Showed superior structural coherence with higher ROUGE-L scores.
- Maintained better n-gram precision reflected in BLEU [8] scores.
- Had the highest success rate with fewer API errors

Gemma-3-27B performed solidly in second place, while Qwen-3-4B, despite being the smallest model, still produced reasonable summaries though with lower metric scores across the board. All models successfully generated Arabic summaries that captured main ideas, though the quality and completeness varied. GPT-OSS-20B's summaries tended to be more comprehensive while remaining concise.

5.2. Question Answering Task Results

We assessed the models' ability to answer questions based on Arabic context passages.

Evaluation Setup:

- **Dataset:** sadeem-ai/arabic-qna.
- **Sample Size:** 1000 examples.
- **Models Tested:** GPT-OSS-20B, Gemma-3-27B, Qwen-3-4B.
- **Evaluation Metrics:** Exact Match (EM), ROUGE-L, F1

Performance Results:

Model	Exact Match	ROUGE-L	F1
GEMMA-3-27B	43.8%	0.615	0.674
GPT-OSS-20B	37.1%	0.578	0.569
Qwen-3-4B	31.3%	0.447	0.412

Figure 14: Question Answering Results

Key Findings:

Gemma-3-27B showed superior question answering performance:

- Achieved the highest Exact Match rate at 43.8%.
- Demonstrated better answer extraction with the strongest ROUGE-L scores.
- Successfully handled both factual and reasoning-based questions more effectively

GPT-OSS-20B came in second with respectable performance, while Qwen-3-4B, as the smallest model, achieved 31.3% exact match—still showing competence despite its size limitations. While GPT-OSS-20B and Qwen-3-4B provided relevant answers, they were often phrased differently from the reference answers or included extra context beyond what was asked. Gemma-3-27B exhibited the best precision in answer extraction, providing concise responses that more closely matched expected answers.

5.3. Sarcasm Detection Task Results

We evaluated the models' ability to recognize sarcasm in Arabic text, including both Modern Standard Arabic and Egyptian dialect.

Evaluation Setup:

- **Dataset:** Arabic Sarcasm Dataset (iabufarha/ar_sarcasm).
- **Sample Size:** 1000 examples.
- **Models Tested:** GPT-OSS-20B, Gemma-3-27B, Qwen-3-4B.
- **Evaluation Metrics:** Accuracy, F1 Score, Recall.

Performance Results:

Model	Accuracy	F1 Score	Precision	Recall
GPT-OSS-20B	87.2%	0.745	0.823	0.681
Gemma-3-27B	82.4%	0.682	0.759	0.618
Qwen-3-4B	76.8%	0.598	0.651	0.553

Figure 15: Sarcasm Detection Results

Key Findings:

Gemma-3-27B demonstrated significantly better sarcasm detection:

- Achieved the highest accuracy at 87.2%.
- Best balanced precision and recall reflected in F1 scores.
- Superior ability to identify sarcastic instances with highest recall.

Gemma-3-27B secured first place with the highest accuracy, while GPT-OSS-20B came in second with solid performance, and Qwen-3-4B showed the expected limitations of a smaller model in this culturally nuanced task. Gemma-3-27B showed stronger capability in understanding cultural context and linguistic cues that signal sarcasm in Arabic. It was particularly effective at catching subtle sarcasm in Egyptian dialect, while the other models sometimes interpreted sarcastic statements literally, especially when cultural context was crucial.

5.4. Comparative Analysis

Overall Model Performance:

Analyzing the results across all three tasks reveals distinct patterns:

Gemma-3-27B:

- **Strengths:** Excelled at tasks requiring cultural understanding and nuanced interpretation (summarization and sarcasm detection).
- **Performance:** Won 2 out of 3 tasks (Summarization: 2nd, QA: 1st, Sarcasm: 1st).
- **Best Use Cases:** Content generation, sentiment analysis, cultural context understanding.

GPT-OSS-20B:

- **Strengths:** Superior at precision-focused tasks requiring exact information extraction (question answering).
- **Performance:** Balanced performer (Summarization: 1st, QA: 2nd, Sarcasm: 2nd)..
- **Best Use Cases:** Information extraction, factual question answering, structured data tasks

Qwen-3-4B:

- **Strengths:** Efficient smaller model with competitive performance given its size.
- **Performance:** Consistent third place across all tasks (Summarization: 3rd, QA: 3rd, Sarcasm: 3rd).
- **Best Use Cases:** Resource-constrained environments, applications prioritizing speed over maximum accuracy.

Task Difficulty Observations:

Based on overall model performance across our 1000-sample evaluation:

1. **Question Answering** (Easiest): Best model (Gemma-3-27B) achieved 43.8% exact match, showing reasonable success.
2. **Summarization** (Moderate): Required balancing content selection and conciseness, with best ROUGE-1 at 0.378.
3. **Sarcasm Detection** (Culturally Challenging): While Gemma-3-27B achieved 87.2% accuracy, this task requires deep cultural understanding and remains challenging for models to interpret correctly, especially with dialectal Egyptian sarcasm.

5.5. Factors Affecting Performance

Our experiments revealed several factors that significantly influence model performance on Arabic NLP tasks:

1. Prompt Engineering:

The way we phrase instructions to models has substantial impact. In our testing, we observed that:

- **Explicit format specifications** helped both models provide better-structured responses.
- **Clear task definitions** reduced ambiguity in outputs.
- **Language consistency** (using Arabic throughout) improved response quality.

Potential for Improvement: With optimized prompts, we expect performance could improve across tasks. Different models respond differently to prompt styles, suggesting that task-specific prompt tuning is valuable.

2. Data Quality and Quantity:

Our 1000-sample evaluation provides initial insights but has limitations:

- **Sample Size:** Larger samples would provide more reliable performance estimates.
- **Data Diversity:** Broader coverage of text types, topics, and complexity levels would reveal more nuanced capabilities.
- **Dialectal Coverage:** More extensive dialect representation would better test cultural understanding.

Expected Impact: Comprehensive evaluation on larger, more diverse datasets would likely reveal:

- More accurate performance baselines.
- Better understanding of model consistency.
- Clearer identification of specific failure patterns.

3. Task-Specific Characteristics:

Different tasks benefit from different model strengths:

Summarization benefits from:

- Strong language generation capabilities.
- Ability to identify key information.
- Understanding of Arabic discourse structure.

Question Answering benefits from:

- Precise information extraction.
- Strong reading comprehension.
- Ability to locate relevant text spans.

Sarcasm Detection benefits from:

- Cultural and contextual awareness.
- Understanding of linguistic markers (exaggeration, contradiction).

- Familiarity with dialectal expressions.

Optimization Opportunities:

The variability in performance suggests several promising directions for improvement:

1. **Prompt Optimization:** Developing task-specific and model-specific prompts could yield significant gains.
2. **Ensemble Approaches:** Combining strengths of different models.
3. **Fine-tuning:** Additional training on Arabic-specific data could improve all tasks.
4. **Dialectal Adaptation:** Specialized handling of dialectal variations could boost sarcasm detection.
5. **Larger Evaluations:** Comprehensive testing on full datasets would validate and refine these initial findings.
6. **Model Selection by Resources:** Qwen-3-4B offers a viable option when computational resources or costs are constrained.

Practical Implications:

For practitioners deploying Arabic NLP systems:

- **Choose models based on task requirements:** GPT-OSS-20B for content understanding and cultural tasks, Gemma-3-27B for information extraction, Qwen-3-4B for resource-efficient applications.
- **Invest in prompt engineering:** Well-crafted prompts can substantially improve results.
- **Consider data characteristics:** MSA vs. dialect, formal vs. informal, domain-specific language all affect performance.
- **Test thoroughly:** Small-scale tests like ours provide direction, but production systems need extensive validation.
- **Combine approaches:** No single model excels at everything—hybrid systems may perform best.
- **Balance performance vs. efficiency:** Smaller models like Qwen may suffice for less critical applications.

Chapter Summary:

Our evaluation on 1000 samples per task revealed that:

- **GPT-OSS-20B** excels at summarization, showing strength in culturally nuanced tasks.
- **Gemma-3-27B** outperforms in question answering and sarcasm detection, demonstrating superior information extraction.
- **Qwen-3-4B** provides a competent, resource-efficient alternative across all tasks.
- **Performance is highly improvable** through prompt optimization, better data, and dialectal adaptation.
- **Current capabilities are promising** but significant room remains for advancement in Arabic NLP.

6. Conclusion

This research sets out to address a critical gap in natural language processing: the need for systematic, accessible evaluation of how open-source multilingual language models handle Arabic text. Through the development of a comprehensive benchmarking framework and empirical testing across three key tasks, we've contributed both practical tools and valuable insights to the Arabic NLP community.

6.1. Summary of Contributions

This project delivers several concrete contributions to Arabic natural language processing research and practice:

1. A Flexible, Open-Source Benchmarking Framework

We developed and released a complete benchmarking system specifically designed for evaluating Arabic language model capabilities. This framework stands out for several reasons:

- **Accessibility:** Built with a user-friendly web interface (Streamlit) that requires no coding knowledge to operate, making it usable by researchers, developers, and educators alike.
- **Modularity:** Designed with clear component separation, allowing easy extension to new tasks, models, or evaluation metrics.
- **Configurability:** Users can test different models, customize prompts, upload their own datasets, and adjust evaluation parameters without modifying code.
- **Reproducibility:** Comprehensive logging and result archiving ensure experiments can be replicated and verified.
- **Open Source:** Freely available with complete documentation, encouraging community adoption and collaborative improvement.

2. Standardized Evaluation Methodology for Arabic NLP

We established a systematic approach to evaluating three critical Arabic NLP tasks:

- **Summarization:** Using ROUGE [7], BLEU [8], and METEOR [9] metrics to assess content coverage, linguistic quality, and semantic preservation.

- **Question Answering:** Employing Exact Match and ROUGE-L metrics, with optional semantic matching for nuanced evaluation and some basic metrics (accuracy and F1).
- **Sarcasm Detection:** Applying classification metrics (accuracy, F1, recall) to measure cultural and contextual understanding.

This methodology provides a template that other researchers can adopt, adapt, or build upon, promoting consistency and comparability across Arabic NLP studies.

3. Empirical Performance Baselines

Through our evaluation of GPT-OSS-20B, Gemma-3-27B, and Qwen-3-4B on 1000 samples per task, we established initial performance baselines for open-source models on Arabic tasks. While limited in scope, these baselines provide reference points for:

- Understanding current model capabilities.
- Identifying which models suit which tasks.
- Measuring future improvements.
- Setting realistic expectations for Arabic NLP applications.

4. Practical Tools and Documentation

Beyond research contributions, we deliver practical resources:

- **Working Code:** A complete, tested implementation that others can use immediately.
- **Comprehensive Documentation:** Clear setup instructions, usage guides, and technical explanations in this documentation and in the README file.
- **Example Workflows:** Demonstrated approaches for running benchmarks and interpreting results.
- **Best Practices:** Guidance on prompt engineering, model selection, and evaluation interpretation.

5. Integration of Advanced Evaluation Techniques

We incorporated optional semantic matching using LLM-as-judge (Gemini), demonstrating how to go beyond traditional metrics to capture meaning equivalence—particularly valuable for Arabic where paraphrasing is common and multiple phrasings can express the same concept.

6.2. Key Findings

Our experiments yielded several important insights about Arabic language model performance:

Model Performance Patterns:

- Task-Specific Strengths:** No single model dominated all tasks. GPT-OSS-20B excelled at summarization, while Gemma-3-27B performed best at question answering and sarcasm detection. This suggests that model architecture, training data, or instruction-tuning approaches affect different capabilities.
- Size Matters, But Not Absolutely:** While larger models (GPT-OSS-20B at 20B, Gemma-3-27B at 27B) generally outperformed the smaller Qwen-3-4B, the performance gap wasn't always proportional to size differences. Qwen demonstrated that smaller models can still provide competent Arabic NLP capabilities, making them viable for resource-constrained applications.
- Cultural Understanding Varies:** Sarcasm detection showed the widest performance variance across models, highlighting that cultural and contextual understanding—essential for Arabic with its rich cultural context and dialectal variations—remains a significant challenge even for capable models.

Performance Improvement Opportunities:

Several factors emerged as high-impact areas for improvement:

- Prompt Engineering:** How we phrase instructions significantly affects outputs. Different prompt strategies could yield performance improvements.
- Data Characteristics:** Models performed better on Modern Standard Arabic than dialectal content, suggesting targeted training or adaptation could narrow this gap.

3. **Sample Size:** Our 1000 sample evaluation provides directional insights, but larger-scale testing would reveal more stable, reliable performance measures.
4. **Evaluation Metrics:** Traditional metrics sometimes miss semantically correct but differently phrased answers, highlighting the value of more flexible evaluation approaches.

Practical Insights:

For those building Arabic NLP applications:

- **Match models to tasks:** Task-specific model selection yields better results than using one model for everything.
- **Invest in prompts:** Well-crafted prompts substantially improve performance at minimal cost.
- **Consider resource trade-offs:** Smaller models may suffice when maximum accuracy isn't critical.
- **Test on representative data:** Performance varies with text type, dialect, and domain.

6.3. Limitations and Constraints

Honest acknowledgment of limitations is essential for proper interpretation of our work:

1. Sample Size Constraints

Our evaluation used 1000 examples per task which is sufficient for initial exploration but limited for definitive conclusions:

- **Statistical Significance:** Small samples mean higher variance; performance estimates have wider confidence intervals.
- **Coverage:** May not fully represent dataset diversity, edge cases, or the full range of difficulty.
- **Generalizability:** Results might not fully predict performance on larger-scale deployments.

Why This Limitation Exists: API rate limits and cost constraints prevented exhaustive evaluation. This is an honest reflection of real-world research constraints when working with commercial APIs.

2. Model Selection

We tested three models, but the open-source landscape includes many others:

- **Incomplete Coverage:** Models like Llama 3.3, Aya, Command-R, and others weren't evaluated.
- **Version Specificity:** Model versions evolve rapidly; our results reflect specific versions at our testing time.
- **Access Constraints:** We used models available through OpenRouter[13] / Groq [16] rather than comprehensive selection.

Implications: Our findings characterize the tested models but shouldn't be extended to all open-source multilingual LLMs.

3. Evaluation Scope

We focused on three specific tasks, but Arabic NLP encompasses much more:

- **Task Coverage:** Didn't evaluate translation, named entity recognition, sentiment analysis, text generation, or many other tasks.
- **Dataset Selection:** Used specific datasets that may not represent all text types or use cases.
- **Metric Limitations:** Traditional metrics have known weaknesses (e.g., not capturing semantic equivalence, penalizing valid paraphrases).

4. Dialectal Coverage

While we included Egyptian and some Gulf dialect in sarcasm detection:

- **Limited Dialect Representation:** Arabic has numerous dialects (Levantine, Gulf, Maghrebi, etc.) we didn't specifically test.
- **MSA Focus:** Summarization and QA primarily used Modern Standard Arabic.

- **Code-Switching:** Didn't evaluate mixed dialect/MSA usage common in real-world Arabic.

5. Technical Constraints

Several practical limitations affected our implementation:

- **API Dependency:** Using OpenRouter[13] / Groq[16] means we're subject to their rate limits, availability, and pricing.
- **No Local Models:** Couldn't test models requiring local deployment due to resource constraints.
- **Sequential Processing:** API limits prevented batch processing, slowing evaluation.
- **Cost Considerations:** Limited the extent of experimentation we could conduct.

6. Cultural and Linguistic Expertise

While we designed our evaluation carefully:

- **Automated Metrics:** Relied primarily on automated metrics rather than native Arabic speaker evaluation.
- **Cultural Nuance:** Some cultural context in sarcasm detection may require deeper cultural expertise to fully assess.
- **Linguistic Variation:** Arabic's rich morphology and regional variations may present subtleties our evaluation didn't fully capture.

7. Temporal Limitations

Language models and the NLP field evolve rapidly:

- **Model Updates:** Models we tested may have newer versions with different performance.
- **Emerging Techniques:** New prompting strategies, evaluation methods, or training approaches continuously emerge.
- **Dataset Evolution:** Datasets may be updated or improved versions released.

6.4. Impact and Significance

Despite these limitations, this work makes meaningful contributions with both immediate and long-term significance:

Immediate Practical Impact:

For Researchers:

- Provides a ready-to-use tool for evaluating Arabic language models without building infrastructure from scratch.
- Establishes baseline performance data for comparison in future studies.
- Demonstrates methodologies that can be adapted to other tasks or languages.
- Offers a template for systematic, reproducible evaluation.

For Developers:

- Enables informed model selection for Arabic NLP applications.
- Provides practical guidance on prompt engineering for Arabic tasks.
- Offers realistic performance expectations to inform project planning.
- Reduces trial-and-error in finding suitable models for specific use cases.

For Educators:

- Creates a teaching tool for demonstrating NLP evaluation concepts.
- Provides hands-on experience with real language models and datasets.
- Illustrates the importance of systematic evaluation in AI development.
- Shows students the practical challenges of working with multilingual AI.

Broader Significance:

1. Advancing Arabic NLP Research

This work contributes to closing the resource gap between Arabic and more extensively studied languages:

- **Visibility:** Highlights Arabic NLP as an active research area deserving attention.

- **Infrastructure:** Provides tools that lower barriers to Arabic NLP research.
- **Community Building:** Open-source release encourages collaboration and collective progress.
- **Standards:** Promotes evaluation consistency across different studies.

2. Promoting Language Equity in AI

By focusing on Arabic:

- **Representation:** Addresses the underrepresentation of Arabic in NLP research.
- **Access:** Helps ensure Arabic speakers benefit from AI advances, not just English speakers.
- **Cultural Preservation:** Supports technology that respects and preserves Arabic linguistic and cultural heritage.
- **Digital Inclusion:** Contributes to reducing digital divides between language communities.

3. Demonstrating Practical Open Science

This project exemplifies open science principles:

- **Transparency:** Complete documentation of methods, results, and limitations.
- **Reproducibility:** Others can replicate our experiments and verify findings.
- **Accessibility:** Free, open-source tools anyone can use.
- **Extensibility:** Designed for others to build upon and improve.

4. Informing Future Development

Our findings provide actionable insights:

- **Model Training:** Highlights where current models struggle, informing training priorities.
- **Dataset Creation:** Identifies needs for better Arabic datasets, particularly dialectal content.

- **Evaluation Methods:** Demonstrates the value of flexible evaluation beyond rigid metrics.
- **Resource Allocation:** Helps prioritize research and development efforts.

Long-Term Potential:

Looking forward, this framework could:

- Evolve into a community standard for Arabic LLM evaluation.
- Expand to cover more tasks, models, and Arabic dialects.
- Integrate new evaluation techniques as the field advances.
- Serve as foundation for commercial Arabic NLP benchmarking services.

6.5. Future Research Trajectory

This work opens numerous pathways for future research and development:

Immediate Extensions:

1. Expanded Evaluation Scale

- **Larger Samples:** Conduct evaluations on hundreds or thousands of examples for statistical robustness.
- **Full Dataset Coverage:** Test across complete datasets rather than samples.
- **Cross-Dataset Validation:** Evaluate same tasks on multiple datasets to assess generalization.

2. Broader Model Coverage

- **Additional Models:** Test more open-source models (Llama 3.3, Aya, Mistral, etc.).
- **Model Versions:** Track performance across model version updates.
- **Specialized Models:** Include Arabic-specific or Arabic-optimized models.
- **Local Deployment:** Test models run locally to avoid API constraints.

3. More Comprehensive Task Set

- **Additional Tasks:** Add named entity recognition, sentiment analysis, text classification, translation.
- **Multi-Task Evaluation:** Assess models across comprehensive task suites.
- **Task Variations:** Test different dataset versions or task formulations.

Medium-Term Directions:

4. Enhanced Dialectal Coverage

- **Multiple Dialects:** Comprehensive evaluation across Levantine, Gulf, Maghrebi, and other dialects.
- **Dialect-Specific Datasets:** Create or integrate datasets for underrepresented dialects, Building on insights from previous work on Arabic dialects [3, 4], future efforts could develop comprehensive dialectal evaluation suites.
- **Code-Switching:** Evaluate handling of mixed dialect and MSA usage.
- **Dialectal Adaptation:** Test techniques for improving dialectal performance.

5. Advanced Evaluation Techniques

- **Human Evaluation:** Incorporate native Arabic speaker assessments.
- **Cultural Competence Testing:** Develop metrics for cultural understanding.
- **Adversarial Testing:** Create challenging test cases that expose model weaknesses.
- **Semantic Evaluation:** Expand LLM-as-judge approaches for more nuanced assessment.

6. Prompt Engineering Research

- **Systematic Prompt Optimization:** Comprehensive testing of prompt variations.
- **Task-Specific Strategies:** Develop optimized prompts for each task type.
- **Model-Specific Tuning:** Identify best prompting practices for each model.
- **Few-Shot Learning:** Explore in-context learning with examples.

7. Fine-Tuning and Adaptation

- **Arabic Fine-Tuning:** Test on how additional Arabic training improves performance.
- **Task-Specific Adaptation:** Fine-tune models for specific Arabic NLP tasks.
- **Low-Resource Techniques:** Explore parameter-efficient fine-tuning for Arabic.
- **Domain Adaptation:** Adapt models to specific domains (medical, legal, etc.).

Long-Term Vision:

8. Community-Driven Platform

- **Collaborative Benchmarking:** Build a platform where researchers contribute results.
- **Leaderboards:** Public rankings of model performance on standardized tests.
- **Shared Resources:** Community repository of prompts, datasets, and insights.
- **Continuous Evaluation:** Ongoing tracking of model improvements over time.

9. Multilingual Extension

- **Other Languages:** Adapt framework for other underrepresented languages.
- **Cross-Lingual Comparison:** Compare Arabic performance with other languages.
- **Transfer Learning:** Study how multilingual models balance different languages.
- **Language-Specific Challenges:** Identify unique challenges across language families.

10. Real-World Application Testing

- **Production Deployment:** Test models in actual applications with real users.
- **User Studies:** Evaluate how Arabic speakers perceive model outputs.
- **Application-Specific Benchmarks:** Develop tests for specific use cases (education, customer service, etc.).
- **Performance-Cost Analysis:** Comprehensive evaluation of accuracy vs. resource trade-offs.

11. Theoretical Understanding

- **Failure Analysis:** Deep investigation of why models fail on specific examples.
- **Linguistic Analysis:** Study how Arabic language features affect model performance.
- **Cultural Factors:** Research role of cultural knowledge in language understanding.
- **Model Interpretability:** Understand what models learn about Arabic.

Research Questions to Pursue:

- How do different training data compositions affect Arabic capabilities?
- What Arabic-specific architectural modifications improve performance?
- Can we develop Arabic-aware tokenization strategies?
- How does multilingual training trade off with Arabic-specific performance?
- What evaluation metrics best capture Arabic language understanding?
- How can we efficiently adapt large models to dialectal Arabic?
- What role does cultural knowledge play in various NLP tasks?

Call to Action:

We invite the research community to:

- **Use the Framework:** Apply it to your own research questions.
- **Contribute:** Submit improvements, bug fixes, and extensions.
- **Share Results:** Publish your findings to build collective knowledge.
- **Collaborate:** Join efforts to advance Arabic NLP.
- **Extend:** Adapt the framework for new tasks, languages, or approaches.

Closing Reflection:

This project represents one step in the ongoing journey toward more inclusive, effective, and culturally aware AI systems. Arabic, spoken by hundreds of millions of people with a rich literary and cultural tradition, deserves the same quality of NLP tools available for more commonly studied languages.

By providing both tools and insights, we hope to accelerate progress in Arabic NLP research and development. Future work could expand on approaches explored in previous surveys [6] and benchmarks [1, 2]. The framework we've built is not an endpoint but a starting point which is a foundation upon which the community can build toward increasingly sophisticated and culturally competent Arabic language AI.

The path forward requires continued research, development, and collaboration. We're optimistic that through collective effort, the gap between Arabic NLP capabilities and those of more extensively studied languages will continue to narrow, ultimately benefit the global Arabic-speaking community with AI tools that understand and respect their language and culture.

7. References

- [1] Abdelali, A., Mubarak, H., & Chowdhury, S. A. (2023). LARA-Bench: Benchmarking Arabic AI with Large Language Models. *arXiv preprint arXiv:2305.14982*.
- [2] Almazrouei, E., Cojocaru, R., & Baldo, M. (2023). ALGhafa Evaluation Benchmark for Arabic Language Models. In *Proceedings of ArabicNLP 2023*.
- [3] Berrimi, M. (2024). Deep Models for Understanding and Generating Textual Arabic Data. *University of Setif Thesis*.
- [4] Khaled, S., Mohamed, E. H., & Medhat, W. (2024). Evaluating Large Language Models for Arabic Sentiment Analysis. *Procedia Computer Science*, 222, 45-50.
- [5] Khondaker, M. T. I., Waheed, A., & Nagoudi, E. M. B. (2023). GPTaraEval: A Comprehensive Evaluation of ChatGPT on Arabic NLP. *arXiv preprint arXiv:2305.14976*.
- [6] Mashaabi, S., Al-Khalifa, S., & Al-Khalifa, H. (2024). A Survey of Large Language Models for Arabic Language and its Dialects. *arXiv preprint arXiv:2410.20238*.
- [7] Lin, C. Y. (2004). ROUGE: A Package for Automatic Evaluation of Summaries. In *Text Summarization Branches Out* (pp. 74-81).
- [8] Papineni, K., Roukos, S., Ward, T., & Zhu, W. J. (2002). BLEU: A Method for Automatic Evaluation of Machine Translation. In *Proceedings of ACL* (pp. 311-318).
- [9] Banerjee, S., & Lavie, A. (2005). METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments. In *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization* (pp. 65-72).
- [10] Shahrour, A. (2023). Arabic Text Summarization Dataset. *Hugging Face Datasets*. Retrieved from <https://huggingface.co/datasets/abdalrahmanshahrour/ArabicTextSummarization>
- [11] Sadeem AI. (2024). Sadeem Arabic QnA Dataset. Hugging Face Datasets. Retrieved from <https://huggingface.co/datasets/sadeem-ai/arabic-qna>
- [12] Abu Farha, I. (2020). Arabic Sarcasm Dataset. *Hugging Face Datasets*. Retrieved from https://huggingface.co/datasets/iabufarha/ar_sarcasm

-
- [13] OpenRouter. (2024). OpenRouter API Documentation, <https://openrouter.ai/docs>
- [14] Google. (2024). Gemini API Documentation, <https://ai.google.dev/docs>
- [15] Streamlit. (2024). Streamlit Documentation, <https://docs.streamlit.io>
- [16] Groq. (2024). Groq API Documentation, <https://groq.com/docs>